Paper Presentation: MegaBlocks: Efficient Sparse Training with Mixture-of-Experts

Eungseo Kim

Seoul National University, Biointelligence Lab

October 28, 2025

Original Paper Information

Original Authors

Trevor Gale[1], Deepak Narayanan[2], Cliff Young[3], Matei Zaharia[1]

Publication

MLSys 2023 - Proceedings of Machine Learning and Systems

Affiliations

[1] Stanford University, Stanford, California, USA
[2] Microsoft Research, Redmond, Washington, USA
[3] Google Research, Mountain View, California, USA

Table of Contents

1. TL;DR
2. Introduction
3. Content
4. Discussion
5. References

TL;DR

The Problem

- Dynamic routing leads to uneven token distribution across experts
- Existing approach: Drop excess tokens, pad insufficient experts

MegaBlocks Solution

- Never drop tokens!
- Adapt block-sparse operations
- No performance loss, significant speedup

What is Mixture-of-Experts (MoE)?

Definition

MoE is an architecture that combines multiple expert models to achieve high capacity efficiently by routing each input to a small subset of experts.

Key Components

- **M experts**: neural networks specializing in different sub-problems
- **Router**: decides which experts to consult for each input
- **Gating network**: computes mixture weights for experts; typically linear

MoE Architecture Details

Mathematical Formulation

For each token x (or batch X), MoE routing works as follows [Shazeer et al., 2017, Fedus et al., 2022]:

- Router computes logits: $z_m = x^T \theta^{(m)}$ for each expert $m \in [M]$, where $\theta^{(m)} \in \mathbb{R}^d$ is the router parameter for expert m
- Gate probabilities: $p = \text{softmax}(z)$
- Top-k selection: choose k experts with highest probabilities
- Expert networks $\{\phi_m\}_{m=1}^{M}$ (typically FFNs) process routed tokens
- Output: weighted combination of selected experts

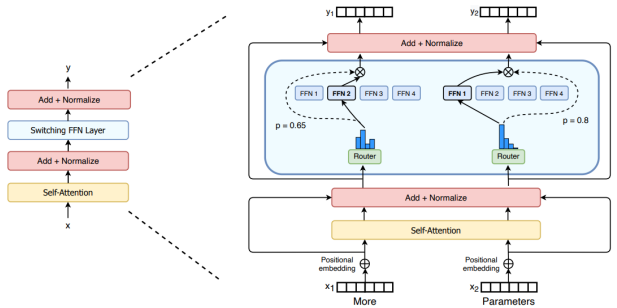


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = "More" and x_2 = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Figure 1: Switch Transformer architecture.

MoE Architecture in MegaBlocks

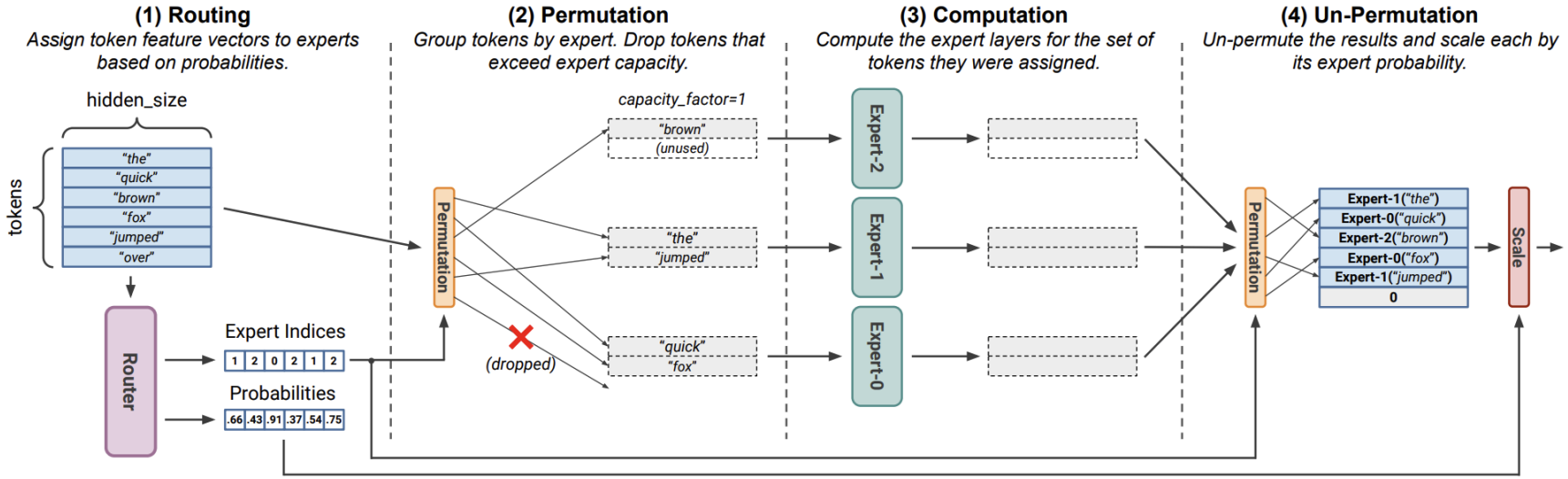


Figure 1. **A Mixture-of-Experts Layer.** Shown for *num_experts=3*, *top_k=1* and *capacity_factor=1* with the prevalent, token dropping formulation. **First (1)**, tokens are mapped to experts by the router. Along with expert assignments, the router produces probabilities that reflect the confidence of the assignments. **Second (2)**, the feature vectors are permuted to group tokens by expert assignment. If the number of tokens assigned to an expert exceeds its capacity, extra tokens are dropped. **Third (3)**, the expert layers are computed for the set of tokens they were assigned as well as any padding needed for unused capacity. **Lastly (4)**, the results of the expert computation are un-permuted and weighted by the router probabilities. The outputs for dropped tokens are shown here set to zero.

Figure 2: MoE architecture introduced by MegaBlocks.

What is Sparsity?

Conditional Computation

Sparsity uses the idea of **conditional computation**. While in dense models all parameters are used for all inputs, sparsity allows us to only run some parts of the whole system.

Scaling Without Computation Overhead

Shazeer's work showed that conditional computation allows scaling model size without increasing computation proportionally—thousands of experts can be used in each MoE layer.

Challenges

- **Uneven batch sizes**: If 10 tokens are batched, 5 might go to one expert and 5 to different experts
- **Underutilization**: This leads to uneven batch sizes and inefficient hardware usage

Gating Mechanism

Learned Gating Network

A learned gating network G decides which experts E to send input to:

$$y = \sum_{i=1}^{n} G(x)_i E_i(x)$$

Efficiency Through Sparsity

When $G(x)_i = 0$, we don't need to compute $E_i(x)$, saving computation.

Simple Gating Function

Traditional setup uses a simple network with softmax:

$$G_\sigma(x) = \text{Softmax}(x \cdot W_g)$$

Noisy Top-k Gating

Adding Noise for Load Balancing

Shazeer's work introduced Noisy Top-k Gating with tunable noise:

1. **Add noise:**

$$H(x)_i = (x \cdot W_g)_i + \text{StandardNormal}() \cdot \text{Softplus}((x \cdot W_{\text{noise}})_i)$$

2. **Keep top k:**

$$\text{KeepTopK}(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in top } k \text{ elements} \\ -\infty & \text{otherwise} \end{cases}$$

3. **Apply softmax:**

$$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k))$$

Why Top-k and Noise?

Benefits of Low k

By using a low k (e.g., 1 or 2), we can train and run inference much faster than if many experts were activated.

Why More Than One Expert?

Initial conjecture: routing to more than one expert was needed for the gate to learn how to route to different experts. Switch Transformers later revisited this decision.

Why Add Noise?

Noise is added for **load balancing** across experts!

Prior MoE Transformer Training

Fixed Expert Capacity Constraint

Prior work defines **fixed expert capacity** [Fedus et al., 2022]:

- **Token dropping**: Excess tokens dropped
- **Padding**: Insufficient experts padded
- Static buffer allocation → inefficiency

Expert Capacity Formula

$$\text{Expert Capacity} = \frac{\text{num tokens}}{\text{num experts}} \times \text{capacity factor}$$

- Capacity factor: multiplier on uniform distribution
- Tradeoff: computation vs. quality
- Higher factor → less dropping, more padding

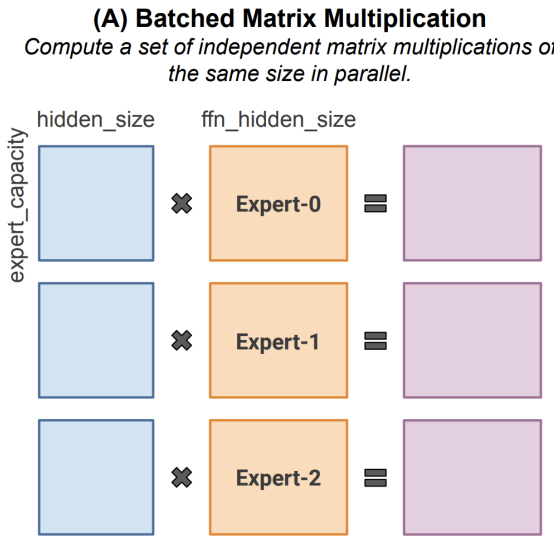


Figure: Expert capacity in Switch Transformer.

Expert Capacity Factor

Impact of Token Dropping

Token dropping significantly impacts model quality:

- Capacity factor = 1: 0.15 validation loss reduction
- Avoiding token dropping: 0.26 reduction (**1.73× larger**)
- Higher capacity enables better performance than dense models

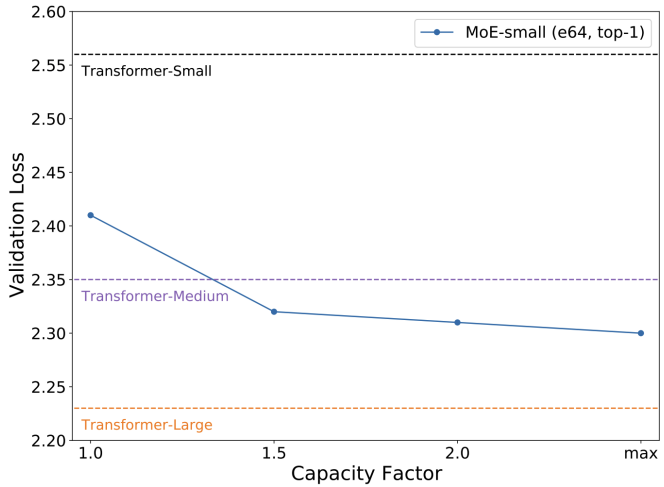
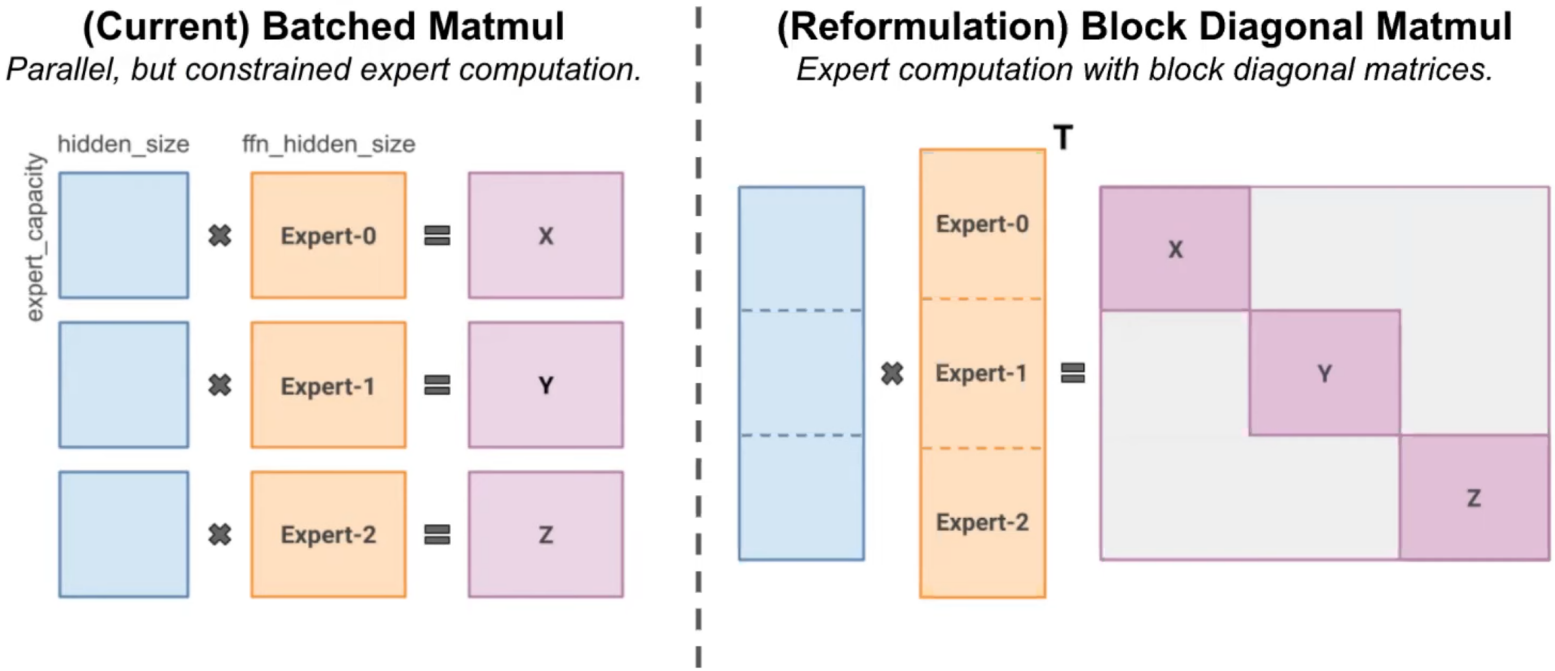


Figure 2. **MoEs Trained on The Pile with Different Capacity Factors.** The loss reached by the MoE models decreases significantly as expert capacity is increased, but at the cost of additional computation. The lowest loss is achieved by the "max" capacity factor model, which avoids dropping tokens through the dynamic capacity factor mechanism proposed by Hwang et al. (2022).

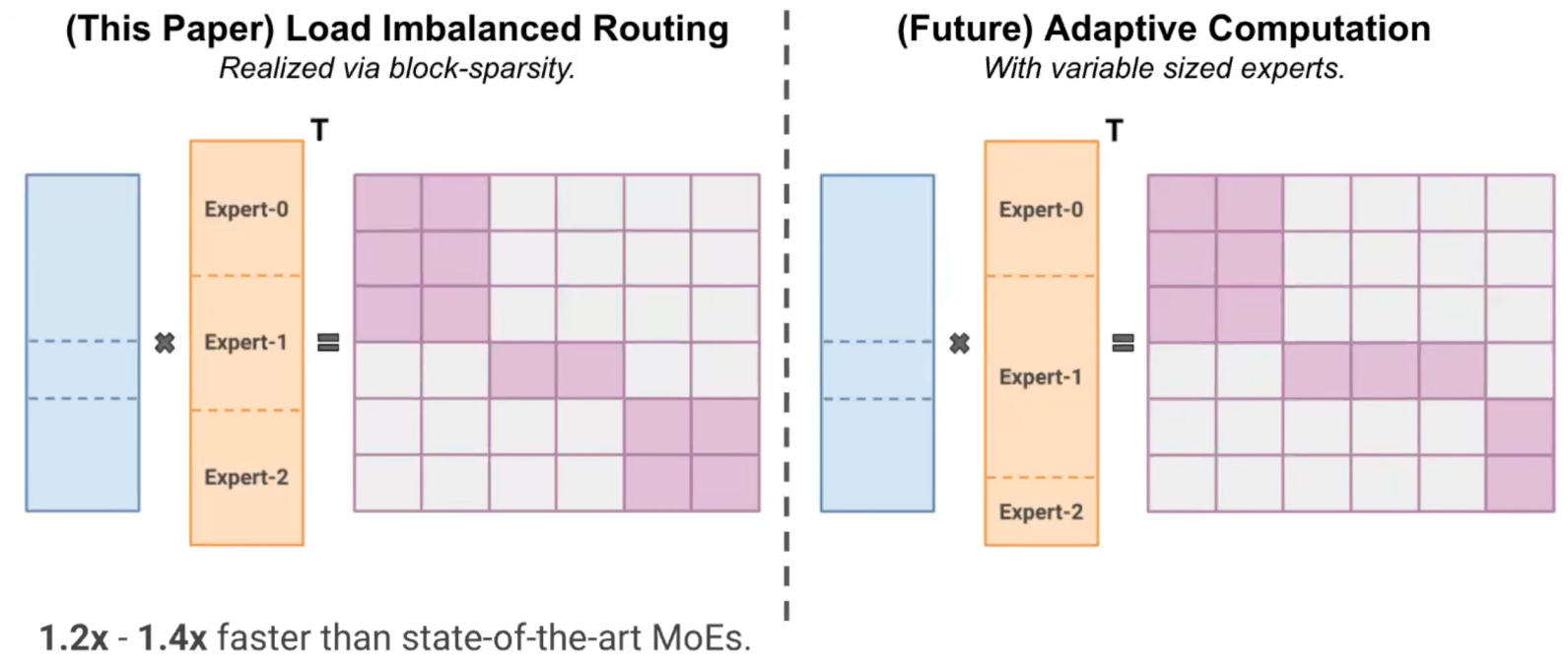
Figure: Effect of capacity factor on model quality.

Idea: Block Diagonal Reformulation (1)



MegaBlocks reformulates MoE computation as block-diagonal matrix operations, enabling efficient GPU implementation without token dropping.

Idea: Block Diagonal Reformulation (2)



Handling load-imbalanced routing through block-sparse operations allows dynamic token assignment without capacity constraints.

Introduction

- **Mixture-of-Experts (MoE)** models scale model capacity without proportional increase in computation
- Current frameworks restrict dynamic routing to satisfy software/hardware constraints
- This forces a tradeoff between model quality and hardware efficiency
- Users must choose between dropping tokens or wasting computation on padding

MegaBlocks Approach

Key Innovation

Reformulate MoE computation in terms of **block-sparse operations**

New Block-Sparse GPU Kernels

Efficiently handle the dynamism present in MoEs

Benefits

- Never drops tokens
- Maps efficiently to modern hardware
- Enables significant speedups

Preliminaries: Sparse Matrix Product Notation

Notation System

Borrowed from Triton (Tillet et al., 2019) to describe sparse/dense matrix multiplication operations

Three-Character String Format

- Each character is either **S** (Sparse) or **D** (Dense)
- Order: **Output** $\rightarrow$ **Left Input** $\rightarrow$ **Right Input**

Examples

- **SDD**: Sparse output, Dense $\times$ Dense inputs
  - Also known as Sampled Dense-Dense Matrix Multiplication (SDDMM)
- **DSD** vs **DDS**: Different forms of Sparse Matrix-Dense Matrix Multiplication (SpMM)
- $\mathbf{SDD^T}$: SDD with transposed right-hand input

Conclusion

- MegaBlocks presents an efficient system for MoE training on GPUs
- Block-sparse operations eliminate the quality-efficiency tradeoff
- Significant performance improvements over existing frameworks
- Enables scalable MoE training without token dropping

Discussion Question

Hardware Trend vs. MoE Direction

MoE models use less computation relative to their model size. However, on modern GPUs, computation is becoming relatively cheap while memory is increasingly expensive.

Key Question

This seems opposite to hardware trends—does this direction still make sense?

References I

Fedus, W., Zoph, B., and Shazeer, N. (2022).
Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity.
Journal of Machine Learning Research.
JMLR.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. (2017).
Outrageously large neural networks: The sparsely-gated mixture-of-experts layer.
In *International Conference on Learning Representations*.

Thank you!

juniormoo@snu.ac.kr